REPAIRFLOW AI

AI-Powered Repair Intelligence & Shop Operations Platform

Prepared as an enhanced proposal in response to feedback requesting greater product value and technical depth.

1. Executive Summary

It brings repair tickets, technician notes, spare-parts inventory, customer communication, operational alerts, and business reporting into one system.

The enhanced product moves beyond basic record keeping by helping technicians investigate device faults, helping managers resolve blocked or overdue repairs, forecasting inventory needs from available history, and answering shop-management questions using verified database records. AI outputs remain recommendations: technicians and authorized staff review and approve consequential decisions.

The proposed stack is React (Vite), Node.js with Express, MySQL, and the Gemini API. Telegram Bot API and Google Sheets

API are supporting integrations. MySQL remains the system of record; integrations must not prevent a valid repair or inventory transaction from being saved.

Product direction: deliver one reliable, end-to-end AI diagnosis and workflow experience first, then expand into inventory intelligence and a database-grounded shop-manager copilot.

2. Problem Statement and Product Opportunity

Repair shops often track jobs manually across notebooks, messaging apps, and spreadsheets. This makes it harder to maintain a consistent repair history, explain delays, remember follow-ups, manage parts, and understand profitability.

- ✓ Technicians may have symptoms and measurements but lack a structured process for recording diagnostic reasoning and next tests.
- ✓ Owners may discover a delayed job only after a customer asks for an update.
- ✓ Basic low-stock thresholds do not account for actual usage, upcoming workload, or supplier lead time.
- ✓ Business questions require manually reviewing repair and sales records.
- ✓ Automated messages and AI recommendations need controls so that incorrect or unapproved content is not sent to customers.

Product opportunity: combine repair-shop workflow data with cautious AI assistance and rule-based automation. The differentiator is not simply adding a chatbot; it is connecting recommendations to real repair records, clear next actions, approval controls, and an auditable history.

3. Target Users and Jobs to Be Done

User	Primary needs
Shop owner / manager	Understand workload, delays, stock risk, revenue and margin; assign responsibility and review important actions.
Repair technician	Record symptoms and measurements, receive diagnostic suggestions, document tests and outcomes, and track parts used.
Front-desk staff	Create tickets, capture customer contact and notification consent, share approved updates, and follow up on approvals.
Customer (limited access)	Receive clear repair-status updates and approval requests through a configured, consented channel.

4. Product Goals and Differentiators

- ✓ AI-assisted diagnosis: transform symptoms and technician-provided evidence into possible causes, safe diagnostic checks, and parts to inspect—not unsupported certainty.
- ✓ Actionable workflow automation: detect overdue or blocked repairs, identify known blockers, create tasks, assign responsibility, and prevent duplicate alerts.
- ✓ Evidence-grounded shop copilot: answer business questions from queried database records and show the records or metrics behind each answer.
- ✓ Inventory intelligence: combine on-hand quantity, reorder threshold, consumption history, and known repair demand to produce explainable restock suggestions when enough data exists.
- ✓ Traceability and human control: save AI suggestions, staff decisions, status changes, task outcomes, and integration failures.
- ✓ Operational reliability: keep core workflows usable when Gemini, Telegram, or Google Sheets is unavailable.

These features are intended to create a credible differentiator from a basic CRUD repair tracker. They are proposed capabilities and should be described as implemented only after they are built and tested.

5. Scope and Delivery Priorities

Priority	Scope	Acceptance outcome
P0 — Core release	Authentication and roles; repair tickets and history; AI diagnostic suggestions with review; parts usage; overdue/blocked-work queue; low-stock rules; audit trail.	A complete repair workflow works from the React UI through Express to MySQL. AI suggestions are saved only after staff approval. Actionable blockers are visible and duplicate alerts are avoided.
P1 — Intelligence	Database-grounded shop-manager copilot; inventory usage trends and explainable restock suggestions; technician workload views; daily operational digest.	Answers use verified database results, disclose missing data, and link to relevant records or metrics.
P2 — Integrations and expansion	Telegram notifications, consent-aware customer updates, Google Sheets reporting, retry controls, richer analytics, and optional supplier lead-time support.	Integration failures are logged and retriable without losing the primary business record.

6. Core User Workflows

6.1 AI-assisted repair diagnosis

1. Staff opens a repair ticket and records device brand/model, reported symptoms, previous repair attempts, and relevant measurements or test results.
2. The backend validates and minimizes the information sent to Gemini, then requests a structured response.
3. The UI presents possible causes, supporting evidence, uncertainty, diagnostic steps, safety cautions, and potentially relevant parts as suggestions.
4. The technician can edit, accept, reject, or regenerate the suggestions and record test results.

5. Only staff-approved information is added to the official diagnosis or repair plan; the system records who approved it and when.

6.2 Intelligent repair workflow

1. A scheduled process and relevant status-change events evaluate rules such as promised date passed, waiting for a part, awaiting customer approval, or no recent progress.
2. The system creates or updates an actionable task with a reason, priority, owner, due date, and recommended next step where supported by data.
3. Staff reviews the task, acts, and records the outcome. Notifications are deduplicated using stable event keys.
4. The dashboard shows unresolved blockers and overdue jobs, while the audit trail records rule execution and staff actions.

6.3 Database-grounded AI shop manager

1. The owner asks a natural-language question, such as “Which repairs are overdue?” or “Which parts have been used most this month?”
2. The backend maps the request to a limited set of authorized metrics or parameterized query templates; the model must not receive unrestricted database credentials or execute arbitrary SQL.
3. The server retrieves the relevant records, calculates metrics where possible, and gives the model only the necessary results to compose an answer.
4. The UI displays the answer with its date range, relevant counts or records, and a clear indication when data is insufficient.

6.4 Inventory intelligence

1. Staff records products, SKU, quantity, unit cost, selling price, reorder threshold, and stock movements.
2. The system flags items at or below their reorder threshold and tracks usage from recorded repair-parts transactions.
3. When enough history exists, it estimates near-term demand and recommends a restock quantity with the basis of the recommendation shown.
4. Staff confirms stock adjustments and purchase decisions; predictions do not silently change inventory.

6.5 Customer communication

1. When a repair status changes, the system prepares a concise draft update.
2. Staff reviews the content. Customer messages are sent only when contact details, channel configuration, and required consent are present.
3. Delivery outcomes are stored; failures can be retried without duplicating the repair transaction.

7. Functional Requirements

ID	Requirement	Acceptance criteria
FR-01	Repair-ticket management	Create, view, edit, search, filter, and paginate repair tickets; persist records in MySQL.
FR-02	Repair lifecycle and history	Support Received, Diagnosing, Waiting for Part, Waiting for Customer, In Repair, Ready for Pickup, Completed, and Cancelled. Store old/new status, timestamp, user, and note.
FR-03	AI diagnosis assistant	Return schema-validated suggestions including possible causes, evidence, diagnostic checks, uncertainty, safety cautions, and suggested parts when relevant. Staff approval is required.
FR-04	Diagnosis and test history	Store approved diagnosis, technician test results, parts inspected/replaced, and outcome against the repair ticket.
ID	**Requirement**	**Acceptance criteria**
FR-05	Workflow rules and tasks	Detect overdue and blocked jobs; create actionable tasks with reason, owner, priority, and resolution state.
FR-06	Inventory and stock movements	Manage products and quantities; record additions, usage, adjustments, and links to repairs where applicable.
FR-07	Low-stock and demand insights	Identify items at/below reorder threshold. Show demand estimates only when sufficient history is available and explain the basis.
FR-08	AI shop-manager copilot	Answer approved business questions from server-retrieved records or metrics; restrict access by role and disclose missing data.

FR-09	Customer notifications	Prepare status drafts and send only through configured channels when consent and destination requirements are met.
FR-10	Scheduled automation	Run periodic overdue/stock checks, deduplicate events, and record run outcomes.
FR-11	Business dashboard	Show open, overdue, blocked, ready-for-pickup, low-stock, and selected revenue/cost metrics with clear date ranges.
FR-12	Integrations and retries	Track Telegram and Google Sheets outcomes, errors, attempts, and retry state without losing core records.
FR-13	Audit and observability	Record important staff actions, AI suggestion approval/rejection, workflow runs, and integration failures.
FR-14	Search and filtering	Support practical filters for status, date, assigned technician, device model, blocker, and inventory category.

8. Non-Functional Requirements

- ✓ Security: store credentials and API keys on the backend; hash passwords; use authentication, role-based authorization, and least-privilege access.
- ✓ Input validation: validate request bodies on the server, use parameterized SQL, enforce allowed status transitions, and return useful error messages.
- ✓ AI reliability: require structured outputs, validate every field, handle timeouts and malformed responses, and label AI content as advisory.
- ✓ Data privacy: send only necessary data to the model; avoid unnecessary customer identifiers and sensitive device data; define retention and deletion practices.
- ✓ Reliability: core MySQL transactions must succeed independently of optional external integrations; use retries with back off and idempotency/deduplication.
- ✓ Maintainability: separate routes, controllers, services, database access, AI integration, automation jobs, and notification adapters.
- ✓ Usability: responsive desktop/mobile UI, loading and empty states, accessible labels, clear uncertainty and approval states, and recoverable errors.
- ✓ Performance: paginate list endpoints, index frequently filtered fields, and avoid expensive queries on every dashboard render.
- ✓ Testing: include unit tests for business rules, integration tests for API/database workflows, and end-to-end tests for critical user journeys.

9. Proposed Architecture

React (Vite) Express REST API MySQL (source of truth). Express coordinates authentication, validation, business services, approved query templates, AI calls, workflow rules, and integrations.

Layer	Responsibilities
React client	Repair workspace, diagnosis review, task queue, inventory, dashboard, copilot UI, integration status.
Express API	Authentication, authorization, validation, business rules, request limits, consistent API responses.
MySQL	Users, customers, repairs, status history, diagnosis records, products, stock movements, tasks, audit events, notifications, integration logs.
AI service	Gemini calls for structured diagnostic suggestions and language generation; no direct database credentials.
Layer	**Responsibilities**
Workflow service / scheduled jobs	Rule evaluation, overdue/blocked detection, low-stock checks, deduplication, task creation.
Integration adapters	Telegram messages and Google Sheets exports with logging, retries, and configuration isolation.

Recommended request pattern: validate and authorize commit the primary database transaction write an event/outbox record process optional AI or integration work record result and retry state. Use transactions for related database updates.

An outbox-style pattern is recommended if reliable background delivery is needed.

10. Proposed Data Model

Entity	Key fields / purpose
Users	id, name, email, password hash, role, created at
Customers	id, name, phone, telegram_chat_id nullable, notification consent, created_at
Repairs	id, ticket_number, customer_id, device_brand, device_model, reported_issue, diagnosis_summary, status, estimated_cost, promised_at, assigned_to, created_at, updated_at
RepairStatusHistory	id, repair_id, old_status, new_status, changed_by, note, created_at
DiagnosisSessions	id, repair_id, model/provider, input_summary, structured_suggestions, uncertainty_notes, status, reviewed_by, reviewed_at, created_at
DiagnosticTests	id, repair_id, diagnosis_session_id nullable, test_name, measurement, result, technician_id, created_at
Products	id, sku, name, category, quantity_on_hand, reorder_level, unit_cost, selling_price, updated_at
StockMovements	id, product_id, repair_id nullable, movement_type, quantity_delta, unit_cost, reason, created_by, created_at
WorkflowTasks	id, repair_id nullable, task_type, reason, priority, assigned_to, due_at, status, deduplication_key, resolved_at
AuditEvents	id, actor_id nullable, entity_type, entity_id, action, metadata, created_at
Notifications	id, repair_id nullable, channel, recipient, message, status, error_message, attempts, sent_at, created_at
IntegrationSyncLog	id, event_type, entity_id, destination, status, attempts, last_error, next_retry_at, created_at

Use foreign keys, unique constraints for ticket numbers and product SKUs, indexes for common filters (status, promised date, assigned technician, product SKU), DECIMAL for money, and consistent timestamp handling. Store stock movements rather than relying only on manual quantity edits. Store only the minimum diagnostic and customer data needed.

11. Proposed API Outline

Method / endpoint	Purpose
GET /api/dashboard/summary	Return authorized KPI counts and current alerts.
GET, POST /api/repairs	List/filter repairs and create a repair ticket.
GET, PUT /api/repairs/:id	Read and update an authorized repair.
PATCH /api/repairs/:id/status	Validate status transition and record status history.

Method / endpoint	Purpose
POST /api/ai/diagnose	Generate structured diagnostic suggestions for a repair.
POST /api/diagnoses/:id/review	Accept, edit, or reject a diagnosis suggestion and record the reviewer.
POST /api/repairs/:id/tests	Record a diagnostic test and result.
GET, POST, PUT /api/products	List, create, and update products.
POST /api/stock-movements	Record stock received, used, or adjusted.
GET /api/workflows/actionable	List overdue and blocked work requiring action.
POST /api/workflows/tasks/:id /resolve	Record task resolution.
POST /api/ai/shop-query	Answer an authorized business question using server-retrieved metrics or records.
GET /api/notifications	View notification delivery outcomes and failures.
POST /api/integrations/telegram/test	Send a test message to a configured authorized shop chat.
POST /api/integrations/sheets/sync	Retry or sync approved pending reporting events.

All endpoints require server-side validation and authorization. Apply pagination and rate limits where appropriate. Do not expose unrestricted SQL execution through the AI copilot.

12. AI Design, Guardrails, and Failure Handling

AI diagnosis response

Use a validated JSON contract with fields such as: summary, possible causes (each with rationale and uncertainty), recommended tests, required tools, potentially_relevant_parts, safety notes, missing information, and limitations. Exact field names can be finalized during implementation.

- ✓ The AI must distinguish reported symptoms from verified test results.
- ✓ It must not claim a component is definitely faulty when evidence is insufficient.
- ✓ It should ask for missing measurements or suggest low-risk checks before invasive actions where appropriate.
- ✓ Staff can edit, accept, reject, or regenerate; approval and the final chosen diagnosis are auditable.
- ✓ Do not let the model silently set repair status, inventory quantity, price, or customer-facing messages.

Copilot grounding

- ✓ Support a bounded set of read-only business intents and approved query templates.
- ✓ Compute totals and counts in application code or SQL; use the model primarily to interpret the question and explain returned results.
- ✓ Include date ranges, definitions, and record references in the answer where practical.
- ✓ If data is absent or ambiguous, say so instead of guessing.
- ✓ Enforce role permissions before retrieving records; never rely on the model to enforce access control.

Resilience

- ✓ Set timeouts and bounded retries for external API calls.
- ✓ Log error categories and correlation IDs without logging secrets.
- ✓ Keep the original repair record even if AI, Telegram, or Sheets is unavailable.
- ✓ Deduplicate scheduled alerts and background jobs using stable keys.
- ✓ Provide a clear retry or manual fallback path for failed integrations.

13. Security, Privacy, and Configuration

- ✓ Keep GEMINI_API_KEY, TELEGRAM_BOT_TOKEN, database credentials, and Google service-account credentials on the backend only.
- ✓ Use environment variables or a secrets manager; keep. env and secrets out of version control.
- ✓ Use password hashing, secure session/JWT handling, role checks, and least-privilege database accounts.
- ✓ Use parameterized SQL and validate ownership/permissions on every resource endpoint.

- ✓ Record customer notification consent and respect channel limitations; do not send messages without a valid destination and required consent.
- ✓ Minimize data sent to AI services and document what is sent. Avoid sharing unnecessary names, phone numbers, or sensitive device identifiers.
- ✓ Use synthetic demo data in screenshots and public demonstrations.

Example configuration keys: PORT, DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME, GEMINI_API_KEY,

TELEGRAM_BOT_TOKEN, TELEGRAM_SHOP_CHAT_ID, GOOGLE_SHEET_ID,

GOOGLE_APPLICATION_CREDENTIALS, CLIENT_ORIGIN, JWT_SECRET.

Use placeholders in .env.example, never real secrets.

14. Success Metrics and Acceptance Tests

Measure	Target for the demo / test
End-to-end repair flow	A staff user can create a repair, persist it, reload it, update status, and see status history.
AI diagnosis workflow	For a seeded test case, the API returns schema-valid suggestions; the UI allows edit/accept/reject; only approved content updates the official diagnosis.
Uncertainty handling	A case with insufficient evidence asks for missing information or clearly states uncertainty rather than claiming a definitive fault.
Workflow automation	A seeded overdue or blocked repair produces one actionable task; repeated job runs do not create duplicate alerts.
Inventory	Stock movements update quantities consistently; low-stock items are identified correctly.
Copilot grounding	A supported question returns an answer matching database test fixtures; unsupported questions disclose limitations.
Integration failure	A simulated Telegram/Sheets failure is logged and does not roll back a saved repair; retry state is visible.
Security	Unauthorized users cannot access protected records or manager-only metrics; invalid inputs are rejected.
Usability	Critical screens show loading, empty, success, and actionable error states on desktop and mobile widths.

These are proposed acceptance targets for the demo, not claims of existing performance. Quantitative model accuracy and inventory forecast quality should be measured on a curated test set before being advertised.

15. Implementation Roadmap

Stage	Deliverables	Definition of done
1. Audit and foundation	Review existing code; finalize schema, roles, API contracts, sample data, and error conventions.	App starts reliably; core data model and protected routes are established.
2. Core repair workflow	Repair tickets, status transitions, status history, stock movements, and basic dashboard.	Core workflows persist correctly and pass API/database tests.
3. AI diagnosis	Gemini service, structured output validation, diagnosis review UI, audit records, failure states.	A technician can review and approve suggestions end-to-end.
4. Workflow intelligence	Overdue/blocked rules, task queue, scheduled jobs, deduplication, owner alerts.	Seeded cases create actionable tasks once and record outcomes.
5. Copilot and inventory insights	Approved business-query intents, grounded answers, usage-based stock insights when data allows.	Responses match test data and clearly state limitations.
Stage	**Deliverables**	**Definition of done**
6. Integrations and polish	Telegram, Sheets, retries, responsive UX, documentation, tests, demo script.	Critical scenarios work with integrations configured and gracefully degrade without them.

16. Demo Plan

17. Risks and Mitigations

Risk	Mitigation
AI suggests an incorrect diagnosis	Present multiple possibilities and uncertainty; require technician verification and record tests.
Copilot gives an unsupported number	Ground answers in server-side queries, test against fixtures, and show the date range and source metrics.

Duplicate notifications or tasks	Use idempotency/deduplication keys and track job outcomes.
External API outage or quota limit	Keep core database transactions independent, use bounded retries, and provide manual fallback.
Insufficient inventory history	Use threshold-based alerts first; enable forecasting only when data is adequate and label estimates.
Scope becomes too large	Prioritize P0 end-to-end workflows; defer P1/P2 until the core demonstration is reliable.

18. Conclusion

comes from connecting technician evidence, reviewed AI suggestions, actionable workflow automation, inventory records, and grounded business answers in a reliable and auditable workflow.

The immediate engineering objective is to deliver a high-quality end-to-end AI diagnosis and workflow automation slice. Further intelligence should be added only after the core data model, permissions, testing, and failure handling are dependable.